

Revue et Challenge de l'Equipe Projet

Audit du Chef de Projet - Evaluation, verification et recommandations

Reference : REV-THUM-2026-001

Version : 1.0

Date : Avril 2026

Auteur : Chef de Projet Data

Objet : Revue critique de chaque role, verification de la couverture des missions, identification des lacunes et recommandations d'excellence

1. Objectif de cette revue

Ce document constitue l'audit du Chef de Projet sur l'ensemble des roles de l'equipe Thumalien. Pour chaque role, le Chef de Projet :

- Verifie que les missions assignees sont bien couvertes par le travail realise
- Challenge les choix techniques et methodologiques
- Identifie les points forts et les lacunes
- Recommande les actions correctives pour atteindre le niveau d'excellence

La demarche est constructive : il ne s'agit pas de pointer des fautes, mais de pousser chaque role a son maximum en identifiant les axes de progression.

2. Grille d'evaluation

Chaque role est evalue sur 5 axes, notes de A (excellence) a D (insuffisant) :

Note	Signification
A	Excellence - Depasse les attentes, proactif, anticipe les problemes
B	Conforme - Missions remplies, qualite professionnelle
C	Partiel - Missions partiellement remplies, axes d'amelioration significatifs
D	Insuffisant - Missions non couvertes, risque pour le projet

3. Revue du Data Engineer

3.1 Etat des lieux

Le pipeline de collecte (collect_bluesky.py, 169 lignes) est fonctionnel et a permis de collecter 188 553 posts. Il utilise l'API AT Protocol avec authentification, recherche par 24 mots-cles bilingues, stockage MongoDB avec deduplication par bulk_write, et resilience par backoff exponentiel.

3.2 Evaluation

Axe	Note	Justification
Architecture pipeline	B	Pipeline fonctionnel avec deduplication et resilience. Architecture monolithique (un seul script) sans separation des concerns
Qualite des donnees	B	Metadonnees completes (URI, CID, auteur, metriques d'engagement). Pas de validation de schema formelle
Performance MongoDB	C	Fonctionnel mais pas d'index optimises documentes. Pas d'aggregation pipeline pour le dashboard
Observabilite	C	Logs basiques (print). Pas de monitoring structure (volume/heure, taux d'erreur, latence)
Scalabilite	C	Boucle de polling toutes les 5 min. Pas d'architecture event-driven (Firehose AT Protocol)

Note globale : B-

3.3 Points forts identifies

- Choix de mots-cles pertinents : 12 termes FR + 12 EN couvrant politique, sante, technologie et sujets neutres. Ce mix garantit un corpus equilibre entre potentielles fake news et contenus normaux
- Resilience : backoff exponentiel sur 3 tentatives + rate limiting aleatoire (0.5-1.5s) pour eviter les bans API
- Metadonnees riches : collecte de reply_count, repost_count, like_count qui pourront servir de features dans les versions futures
- Flag ai_processed : permet un traitement incremental sans retraiter les posts deja analyses

3.4 Points de challenge

Challenge 1 - Pourquoi ne pas utiliser le Firehose AT Protocol ?

Le collecteur actuel utilise une boucle de recherche par mots-cles toutes les 5 minutes. Le protocole AT offre un endpoint com.atproto.sync.subscribeRepos (Firehose) qui fournit un flux temps reel de TOUS les posts.

Question : Avez-vous evalue le Firehose ? Le volume serait-il trop important ? Quels seraient les avantages en termes de couverture et de latence ?

Recommandation : Evaluer le Firehose sur une periode de 24h, mesurer le volume, et comparer la couverture avec la methode actuelle.

Challenge 2 - Absence de monitoring structure

Il n'y a pas de metriques accessibles sur la sante du pipeline : combien de posts collectes par heure ? Quel est le taux d'erreur ? Combien de doublons rejetes ? Si le collecteur tombe silencieusement, combien de temps avant detection ?

Recommandation : Implementer un endpoint de health check et un export de metriques (meme un simple fichier JSON mis a jour toutes les heures avec le volume, les erreurs et le dernier heartbeat).

Challenge 3 - Pas de schema de validation

Les documents MongoDB n'ont pas de validation de schema. Un champ manquant ou un type incorrect ne serait pas detecte a l'insertion.

Recommandation : Ajouter une validation de schema MongoDB (`db.createCollection("raw_posts", {validator: ...})`) ou une validation Python avec Pydantic avant insertion.

3.5 Plan d'action recommande

Action	Priorite	Effort	Impact
Ajouter un monitoring de volume (posts/heure, erreurs)	Haute	2 jours	Detecter les pannes silencieuses
Creer des index MongoDB optimises	Haute	1 jour	Performance du dashboard x10
Implementer une validation de schema	Moyenne	2 jours	Qualite des donnees garantie
Evaluer le Firehose AT Protocol	Moyenne	3 jours	Couverture et latence ameliorees
Separer le collecteur en modules (config, ingestion, stockage)	Faible	3 jours	Maintenabilite

4. Revue du ML Engineer / NLP Specialist

4.1 Etat des lieux

Le pipeline expert (`expert_detector.py`, ~1000 lignes) est le composant le plus sophistique du projet. Il combine TF-IDF (30K features, trigrams), 12 features linguistiques, 7 features emotionnelles (MLP PyTorch), et une Regression Logistique calibree. Le modele V2 est entraine sur 145 703 textes provenant de 6 datasets.

4.2 Evaluation

Axe	Note	Justification
-----	------	---------------

Architecture du pipeline	A	Pipeline modulaire, bien structure (DatasetCleaner, LinguisticFeatureExtractor, EmotionFeatureExtractor, ExpertFakeNewsDetector). Separation nette des responsabilites
Qualite des modeles	A	F1 V2 = 0.897 avec calibration sur textes courts. Seuil optimise (0.44). Ablation study rigoureuse (7 conditions)
Gestion des biais	A	Biais Reuters identifie, quantifie (99.2%) et corrige. Biais linguistique corrige par oversampling x3. Domain shift traite par 3 datasets sociaux
Explicabilite	A	explain_prediction() fournit top mots + features linguistiques + mots sensationnalistes. Integre dans le dashboard
Innovation	B	Excellent pipeline classique. Pas encore d'embeddings semantiques (V3 planifie)

Note globale : A

4.3 Points forts identifies

- Identification et correction du biais Reuters : cette decouverte a sauve le projet. Un modele avec F1=0.99 qui classe tout comme FAKE en production est pire qu'inutile. La methodologie d'audit (audit_reuters_leakage()) est exemplaire
- Choix rationnel de LogReg vs RoBERTa : le rapport qualite/performance/empreinte carbone est optimal. F1 comparable, 100x moins d'energie, interpretabilite totale
- Features linguistiques (12) : captent des signaux structurels independants du contenu (caps_ratio, sensationalism_score, lexical_diversity). C'est du feature engineering intelligent
- 47 mots sensationnalistes FR + 17 EN : une ressource precieuse, specifique au domaine
- Pipeline V2 avec 6 datasets : la diversite des sources (articles longs, titres courts, tweets COVID, tweets FR) est une strategie anti-overfitting efficace

4.4 Points de challenge

Challenge 1 - Le F1 de 0.80 sur textes courts est-il suffisant ?

Le F1 de 0.897 global masque une realite : sur les textes < 30 mots (qui sont le cas d'usage principal sur Bluesky), le F1 est de 0.80. Cela signifie que 1 post sur 5 est mal classe.

Question : Avez-vous analyse le type d'erreurs sur les textes courts ? S'agit-il principalement de faux positifs (fiable classe suspect) ou de faux negatifs ? Quel est l'impact sur l'experience utilisateur ?

Recommandation : Produire une matrice de confusion specifique aux textes < 30 mots. Si les faux positifs dominant, le seuil pourrait etre baisse davantage. Si les faux negatifs dominant, les features linguistiques manquent de signal sur les textes courts.

Challenge 2 - Les 47 mots sensationnalistes sont-ils complets et actualisés ?

La liste de mots sensationnalistes (FR: "scandale", "censuré", "complot"... ; EN: "breaking", "shocking", "bombshell"...) est statique. Le vocabulaire de la désinformation évolue rapidement.

Question : Cette liste a-t-elle été validée par un expert en désinformation ? Est-elle mise à jour périodiquement ?

Recommandation : Enrichir la liste par analyse des coefficients TF-IDF les plus discriminants (déjà disponible via `explain_prediction()`). Prévoir une mise à jour semestrielle en collaboration avec des fact-checkers.

Challenge 3 - La calibration du modèle est-elle validée ?

Le pipeline utilise `CalibratedClassifierCV` pour calibrer les probabilités. Mais la calibration a-t-elle été validée sur les textes courts (pas seulement les articles longs) ?

Question : Avez-vous produit un reliability diagram (diagramme de calibration) spécifique aux posts Bluesky ?

Recommandation : Produire un reliability diagram sur 2 000 posts Bluesky. Si la calibration est mauvaise sur les textes courts, recalibrer avec un dataset mixte.

Challenge 4 - Que se passe-t-il avec les textes non FR/non EN ?

Les posts Bluesky contiennent potentiellement des textes dans d'autres langues (portugais, japonais, allemand...). Le pipeline détecte la langue via `langdetect` et traite les textes non FR/non EN comment ?

Question : Quel est le comportement par défaut pour un texte en portugais ? Est-il classé ? Avec quelle fiabilité ?

Recommandation : Documenter explicitement le comportement pour les langues non supportées. Envisager un flag "confiance réduite" pour ces textes.

4.5 Plan d'action recommandé

Action	Priorité	Effort	Impact
Matrice de confusion par longueur de texte	Haute	1 jour	Comprendre les erreurs sur textes courts
Reliability diagram sur posts Bluesky	Haute	1 jour	Valider la calibration en production
Documenter le comportement pour langues non supportées	Haute	0.5 jour	Transparence
Enrichir la liste de mots sensationnalistes	Moyenne	2 jours	Améliorer le <code>sensationalism_score</code>
Prototyper les Sentence-Transformers (V3)	Moyenne	5 jours	Amélioration F1 textes courts

5. Revue du Data Scientist

5.1 Etat des lieux

Le travail de Data Science est visible dans les notebooks 00 (audit qualité), 05 (ablation study), 07 (GridSearch + feature importance) et 08 (intégration V2). L'approche expérimentale est rigoureuse : chaque choix est justifié par des

metriques.

5.2 Evaluation

Axe	Note	Justification
Rigueur methodologique	A	Ablation study 7 conditions, GridSearch 36 combinaisons, analyse feature importance, test sur Bluesky
Analyse exploratoire	B	Notebook 01 (Bluesky) et 00 (audit) sont bien realises. Pas de clustering thematique ni de detection de signaux faibles
Gestion des biais	A	Biais Reuters identifie par analyse exploratoire. Domain shift identifie par test operationnel. Corrections methodiques
Communication des resultats	A	Rapport technique detaille (566 lignes). Choix documentes avec justification
Analyse d'erreurs	B	Feature importance analysee. Pas d'analyse qualitative manuelle des erreurs du modele

Note globale : A-

5.3 Points de challenge

Challenge 1 - Absence d'analyse d'erreurs qualitative

L'evaluation du modele repose sur des metriques agregees (F1, precision, recall). Mais avez-vous lu manuellement les posts mal classes pour comprendre pourquoi le modele se trompe ?

Recommandation : Lire les 50 faux positifs et 50 faux negatifs les plus confiants (ceux ou le modele est le plus sur de son erreur). Categoriser les erreurs : texte trop court, sarcasme, sujet non couvert, langue mal detectee. Cette analyse qualitative vaut plus que 10 points de GridSearch.

Challenge 2 - Pas de test de derive temporelle

Le modele a ete entraine en fevrier 2026. Les posts sont collectes depuis decembre 2025. Mais les sujets d'actualite changent : les fake news de mars 2026 ne ressemblent peut-etre pas a celles de decembre 2025.

Recommandation : Comparer la distribution des scores mois par mois. Si le % suspect augmente progressivement, c'est un signe de concept drift.

Challenge 3 - Le choix du seuil 0.44 est-il robuste ?

Le seuil a ete optimise sur un echantillon de 2 000 posts Bluesky. Cet echantillon est-il representatif ? Le seuil optimal pourrait-il varier selon le sujet ou la langue.

Recommandation : Tester le seuil optimal par sous-population : FR vs EN, politique vs sante, posts courts vs longs. Si le seuil optimal varie significativement, envisager des seuils differencies.

5.4 Plan d'action recommande

Action	Priorite	Effort	Impact
Analyse d'erreurs qualitative (100 posts)	Haute	2 jours	Comprendre les vrais points faibles
Monitoring mensuel de derive	Haute	1 jour/mois	Detecter le concept drift
Test de robustesse du seuil par sous-population	Moyenne	2 jours	Seuil plus precis
Clustering thematique (BERTopic)	Moyenne	3 jours	Detection de signaux faibles

6. Revue du MLOps Engineer

6.1 Etat des lieux

Le projet n'a pas de MLOps formalise. Les modeles sont des fichiers .pkl et .pt dans le dossier models/. Le chargement est fait manuellement par le dashboard via `detector.load(suffix='expert_v2')`. Il n'y a pas de pipeline de retraining automatise, pas de versioning formel des modeles, pas de monitoring de production.

6.2 Evaluation

Axe	Note	Justification
Versioning des modeles	C	Convention de nommage (suffixe _v2) mais pas de registre formel. Pas de hash de dataset associe
Pipeline de retraining	D	Pas de pipeline automatise. Retraining manuel via notebooks
Serving	C	Chargement direct du .pkl dans Streamlit. Fonctionnel mais pas scalable
Monitoring production	D	Aucun monitoring de la derive ou des performances en production
Reproductibilite	B	random_state=42, requirements.txt, Docker. Mais pas de tracking des runs (MLflow/W&B)

Note globale : C-

6.3 Points de challenge

Challenge 1 - Comment savez-vous si le modele V2 est encore performant aujourd'hui ?

Le modele a ete entraine en fevrier 2026. Nous sommes en avril 2026. Les sujets ont change, le vocabulaire a evolue.

Sans monitoring, nous ne savons pas si les 73.4% de fiabilité sur Bluesky tiennent encore.

Recommandation critique : Implementer un monitoring minimal : une fois par semaine, appliquer le modèle sur 1 000 posts récents et enregistrer la distribution des scores. Si le % suspect dépasse 40% de manière durable, déclencher un retraining.

Challenge 2 - Que se passe-t-il si le modèle V2 est corrompu ?

Si le fichier `model_expert_v2.pkl` est corrompu, le dashboard tombe. Le fallback vers V1 existe dans le code, mais le V1 a un biais Reuters et classe 77% des posts comme suspects.

Recommandation : Implementer un test de santé au chargement du modèle : prédire 5 textes de référence et vérifier que les scores sont dans les plages attendues. Si le test échoue, alerter.

Challenge 3 - Pas de tracking des expériences

Les 8 notebooks contiennent des dizaines d'expériences (ablation, GridSearch, comparaisons). Mais il n'y a pas de tracking centralisé. Pour reproduire une expérience ou comparer deux runs, il faut relire les notebooks.

Recommandation : Intégrer MLflow (même en mode local) pour tracker les runs. Chaque entraînement enregistre automatiquement les hyperparamètres, les métriques et les artefacts.

6.4 Plan d'action recommandé

Action	Priorité	Effort	Impact
Monitoring hebdomadaire des scores	Critique	2 jours	Détection de dérive
Test de santé du modèle au chargement	Haute	1 jour	Resilience
Intégration MLflow (tracking local)	Haute	3 jours	Reproductibilité
Pipeline de retraining automatisé (script + cron)	Moyenne	5 jours	Maintenance réduite
API de serving (FastAPI)	Moyenne	5 jours	Scalabilité

7. Revue du Développeur Dashboard

7.1 Etat des lieux

Le dashboard (`dashboard/app.py`, ~1010 lignes) est le composant le plus visible du projet. Il propose 3 pages (Vue Globale, Analyse Temps Réel, Métriques & Transparence) avec un design glassmorphism professionnel, des graphiques Plotly interactifs, et l'explicabilité des prédictions.

7.2 Evaluation

Axe	Note	Justification
Design et UX	A	Glassmorphism cohérent, dark theme avec accents cyan, jauges et radars lisibles

Fonctionnalites	A	3 pages completes, analyse temps reel, explicabilite, metriques de transparence
Accessibilite	B	Contraste WCAG 2.1 AA respecte. Mais pas de support lecteur d'ecran ni ARIA
Performance	B	Caching st.cache_resource et session_state. Pas de pagination pour les gros volumes
Robustesse	A	Mode demo si MongoDB indisponible. Fallback V1 si V2 absent. Normalisation des formats legacy

Note globale : A-

7.3 Points de challenge

Challenge 1 - Le mode demo est-il representatif ?

Le mode demo contient 15 posts pre-definis (8 EN fiables + 7 FR suspects). Cette repartition (53% fiable) est inferieure aux 73.4% observes en production.

Recommandation : Ajuster les posts de demo pour refleter la distribution reelle. Ajouter des posts neutres et des cas limites pour montrer la nuance du modele.

Challenge 2 - Scalabilite du chargement des donnees

Le dashboard charge tous les posts avec `collection.find({}, projection).sort('collected_at', -1).limit(limit)`. Pour 200 000 posts, meme avec un limit, l'aggregation emotionnelle se fait en Python apres chargement.

Recommandation : Deporter les aggregations dans MongoDB (`$group`, `$facet`) et ne charger que les resultats agreges pour la Vue Globale. Charger les posts individuels uniquement pour la page de detail.

Challenge 3 - L'explicabilite est excellente mais pourrait aller plus loin

La section explicabilite montre les top mots et features linguistiques. Mais elle ne montre pas l'impact relatif des 3 groupes de features (TF-IDF vs linguistique vs emotions).

Recommandation : Ajouter un diagramme en barres empilees montrant la contribution de chaque groupe de features au score final. Cela aiderait l'utilisateur a comprendre si la decision est basee sur le vocabulaire, le style ou l'emotion.

7.4 Plan d'action recommande

Action	Priorite	Effort	Impact
Deporter les aggregations dans MongoDB	Haute	3 jours	Performance x5 pour gros volumes
Ajuster les posts de demo	Moyenne	0.5 jour	Representation fidele
Contribution par groupe de features	Moyenne	2 jours	Explicabilite enrichie
Pagination des posts	Moyenne	1 jour	Support de volumes > 10K posts

8. Revue du DevOps / Cloud Architect

8.1 Etat des lieux

L'infrastructure repose sur Docker Compose avec 4 services (MongoDB, Collector, Jupyter, Dashboard). Le Dockerfile est simple (Python 3.9-slim + gcc + requirements). Les volumes assurent la persistance.

8.2 Evaluation

Axe	Note	Justification
Conteneurisation	B	Docker Compose fonctionnel, 4 services bien isolés, volumes persistants
CI/CD	D	Aucun pipeline CI/CD. Pas de tests automatisés au push
Sécurité	C	Secrets dans .env (pas en clair dans le code). Mais MongoDB sans authentification, pas de TLS
Monitoring infra	D	Pas de monitoring. Pas de health checks. Pas d'alertes
Optimisation	C	Image unique pour tous les services (lourde). Pas de multi-stage build

Note globale : C

8.3 Points de challenge

Challenge 1 - MongoDB est sans authentification

Le `docker-compose.yml` lance MongoDB sans `MONGO_INITDB_ROOT_USERNAME` ni `MONGO_INITDB_ROOT_PASSWORD`. Toute personne ayant accès au réseau Docker peut lire/écrire dans la base.

Recommandation critique : Ajouter l'authentification MongoDB, même en local. C'est une bonne pratique de sécurité qui prévient les incidents.

Challenge 2 - Une seule image Docker pour tous les services

Le Dockerfile est le même pour le collecteur, Jupyter et le dashboard. Cela signifie que chaque service embarque toutes les dépendances (PyTorch, Streamlit, Plotly, etc.) même s'il n'en a besoin que d'une partie.

Recommandation : Créer des Dockerfiles spécifiques par service, ou utiliser des multi-stage builds pour réduire la taille des images.

Challenge 3 - Aucun pipeline CI/CD

Il n'y a aucun test automatisé qui se déclenche au push. Un bug dans `expert_detector.py` pourrait être déployé sans détection.

Recommandation : Créer un GitHub Actions minimal : à chaque push, vérifier que le modèle charge, que `predict()` fonctionne sur 5 textes de référence, et que le dashboard démarre.

8.4 Plan d'action recommande

Action	Priorite	Effort	Impact
Ajouter l'authentification MongoDB	Critique	0.5 jour	Securite
GitHub Actions CI basique	Haute	2 jours	Detection automatique des regressions
Health checks dans docker-compose	Haute	1 jour	Auto-restart en cas de panne
Dockerfiles specifiques par service	Moyenne	2 jours	Images 3x plus legeres
Monitoring Prometheus + Grafana	Moyenne	3 jours	Observabilite infrastructure

9. Revue de l'Expert Green IT

9.1 Etat des lieux

CodeCarbon est integre dans le pipeline d'entrainement et les emissions sont enregistrees dans emissions.csv. Le dashboard affiche les metriques Green IT avec des equivalences concretes (km en voiture, emails).

9.2 Evaluation

Axe	Note	Justification
Mesure d'empreinte	A	CodeCarbon integre, emissions mesurees a chaque entrainement
Choix frugaux	A	LogReg vs RoBERTa : 100x moins d'energie. Justification explicite du choix
Communication	A	Equivalences concretes dans le dashboard. Accessible au grand public
Couverture de la mesure	C	Seul l'entrainement est mesure. L'inference, le stockage et le reseau ne sont pas couverts
Optimisation proactive	C	Pas d'objectif de reduction chiffre. Pas de comparaison avec des alternatives

Note globale : B+

9.3 Points de challenge

Challenge 1 - L'empreinte de l'inference n'est pas mesuree

L'entrainement consomme 0.30 g CO₂, mais combien consomme l'inference quotidienne sur 2 000 posts ? Sur une annee, l'inference cumule probablement plus que l'entrainement.

Recommandation : Ajouter CodeCarbon autour de la boucle d'inference dans le pipeline quotidien. Calculer l'empreinte totale annuelle (entrainement + inference).

Challenge 2 - L'empreinte MongoDB n'est pas comptabilisee

MongoDB tourne 24/7. Meme au repos, un serveur consomme de l'energie. Sur une annee, c'est potentiellement 10-50 kWh selon la machine.

Recommandation : Estimer la consommation de base de l'infrastructure Docker (MongoDB + collecteur) et l'integrer dans le bilan carbone total.

9.4 Plan d'action recommande

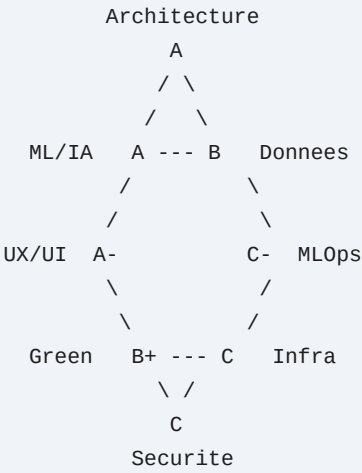
Action	Priorite	Effort	Impact
Mesurer l'empreinte de l'inference	Haute	1 jour	Bilan complet
Estimer l'empreinte infrastructure	Moyenne	1 jour	Vision globale
Definir un budget carbone annuel	Moyenne	0.5 jour	Objectif chiffre

10. Synthese globale - Tableau de bord du Chef de Projet

10.1 Vue d'ensemble des evaluations

Role	Note	Points forts	Risque principal
Data Engineer	B-	Pipeline resilient, metadonnees riches	Pas de monitoring, pas de validation schema
ML Engineer	A	Pipeline expert, gestion des biais, explicabilite	F1 textes courts a ameliorer (0.80)
Data Scientist	A-	Rigueur methodologique, ablation study	Pas d'analyse d'erreurs qualitative
MLOps	C-	Convention de nommage	Aucun monitoring, pas de CI/CD
Dashboard Dev	A-	UX professionnelle, robustesse, explicabilite	Scalabilite pour gros volumes
DevOps	C	Docker fonctionnel	MongoDB sans auth, pas de CI/CD
Green IT	B+	CodeCarbon, choix frugaux, communication	Mesure partielle (entrainement seulement)

10.2 Radar de maturite du projet



10.3 Actions prioritaires (Top 10)

#	Action	Responsable	Priorite	Effort	Impact attendu
1	Authentification MongoDB	DevOps	Critique	0.5j	Securite de base
2	Monitoring hebdomadaire des regressions	MLOps	Critique	2j	Detection de derive
3	CI/CD basique (GitHub Actions)	DevOps	Haute	2j	Detection automatique des regressions
4	Analyse d'erreurs qualitative (100 posts)	Data Scientist	Haute	2j	Comprendre les vrais echecs
5	Test de sante du modele au chargement	MLOps	Haute	1j	Resilience du dashboard
6	Index MongoDB optimises	Data Engineer	Haute	1j	Performance du dashboard
7	Monitoring du pipeline de collecte	Data Engineer	Haute	2j	Detecter les pannes silencieuses
8	Matrice de confusion par longueur	ML Engineer	Haute	1j	Comprendre F1 textes courts
9	Mesure empreinte inference	Green IT	Haute	1j	Bilan carbone complet
10	Aggregations MongoDB pour dashboard	Dashboard Dev	Haute	3j	Performance pour gros volumes

Effort total estimé : ~16 jours-homme pour couvrir les 10 actions prioritaires.

11. Conclusion du Chef de Projet

Ce que ce projet fait remarquablement bien

Thumalien est un projet d'une maturite exceptionnelle pour un contexte academique. Les points suivants meritent d'etre soulignes :

- La decouverte et correction du biais Reuters est un cas d'ecole en data science. Transformer un modele a 99% de F1 (biaise) en un modele a 90% (reel) en comprenant pourquoi est exactement le travail d'un Data Scientist de haut niveau.
- Le choix delibere de LogReg plutot que de deep learning pour la detection de fake news est un choix d'ingenieur mature. Dans un contexte ou tout le monde veut utiliser des Transformers, choisir la simplicité quand elle suffit est un signe d'excellence.
- L'ablation study a 7 conditions depasse le standard academique habituel. Elle prouve methodiquement que chaque composant du pipeline apporte de la valeur.
- L'integration de CodeCarbon et la demarche Green IT ne sont pas du greenwashing : elles guident des choix d'architecture reels (pas de GPU, pas de Transformers en production).
- Le dashboard avec explicabilite est un livrable de qualite professionnelle. L'utilisateur ne voit pas juste un score, il comprend pourquoi.

Ce qui doit progresser pour atteindre l'excellence industrielle

Les lacunes identifiees sont celles d'un projet academique qui n'a pas encore atteint la maturite de production :

- L'absence de monitoring est le risque numero 1. En production, un modele qui derive silencieusement est pire qu'un modele absent, car il inspire une fausse confiance.
- L'absence de CI/CD signifie que chaque changement est un risque non controle. Un test automatise basique (le modele charge ? il predit ? les scores sont dans les plages attendues ?) prend 2 jours a implementer et previent des jours de debug.
- La securite minimale (MongoDB sans auth) est acceptable en local mais inacceptable en production. C'est une dette technique a traiter en priorite.
- L'analyse d'erreurs qualitative manquante est l'opportunité d'amelioration la plus rentable. Lire 100 erreurs du modele revele des patterns invisibles dans les metriques agregees.

Verdict global

Le projet Thumalien est pret pour une mise en production supervisee. Les fondations sont solides (pipeline ML, donnees, dashboard). Les 10 actions prioritaires identifiees dans cette revue permettront de combler les lacunes restantes et d'atteindre un niveau d'excellence industrielle.

Le Chef de Projet recommande de proceder par phases :

- Phase immediate (2 semaines) : Actions 1 a 5 (securite, monitoring, CI/CD)
- Phase court terme (1 mois) : Actions 6 a 10 (performance, analyse)
- Phase moyen terme (3 mois) : Pipeline V3, API REST, monitoring avance

Document redige par le Chef de Projet - Avril 2026

Prochaine revue : Juillet 2026

Reference : REV-THUM-2026-001 - Version 1.0